Lab 2

Instructions for Lab Submission:

1. File Naming and Submission:
    a. Submit your solution as a zip file named **lab2_<ERP>.zip**, where <*ERP*> is your ERP number. For example, if your ERP number is *12345*, the file should be named *lab2_12345.zip*.
    b. Ensure that the zip file contains:
        i. C++ source files named as **task<number>.cpp** (e.g., **task1.cpp**).
        ii. No unnecessary files (such as executables, temporary files, or editor configs).
    c. Do not submit ***TEXT(.txt)*** files.
2. Code Organization:
    a. Write each task in its own **.cpp** file (as specified above).
    b. Ensure that your code is modular and logically separated into functions when specified.
3. Code Neatness:
    a. Ensure your code is well-formatted and easy to read. Use proper indentation and meaningful variable names.
    b. Include appropriate comments to explain the logic where necessary (especially for functions and complex sections of the code).
    c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. Code Compilation and Functionality:
    a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
    b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. Additional Instructions:
    a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
    b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
    c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. Deadline:
    a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted
    b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. Plagiarism Policy:
    a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

Data Types

A data type defines the kind of values a variable can store, the operations that can be performed on it, and how it is represented in memory. Choosing the appropriate type ensures that the variable behaves correctly and that operations on it are meaningful.

Common built-in types in C++:

- Integer types:
  - **`int`**: 32-bit signed integer, range: -2,147,483,648 to 2,147,483,647.
  - **`short`**: 16-bit signed integer.
  - **`long`**: 32-bit signed integer (platform-dependent).
  - **`long long`**: 64-bit signed integer.
- Floating-point types:
  - **`float`** – 32-bit single precision.
  - **`double`** – 64-bit double precision.
  - **`long double`** – typically 80-bit extended precision.
- **`char`**: stores a single character. Internally, it is an integer type, so each character maps to a numeric value according to ASCII (or extended) encoding, e.g., 'a' = 97, 'b' = 98.
- **`bool`**: stores true or false.
- **`std::string`**: stores sequences of characters, e.g., "Hello World".

For a complete table with sizes, ranges, and other built-in types, see: [cppreference.com: C++ fundamental types]

Operators

Operators in C++ allow you to manipulate values, perform calculations, and modify variables directly.

1. Ternary Operator

The ternary operator provides a concise way to select a value based on a condition. Its syntax is:

```
condition ? expression_if_true : expression_if_false;
```

If the condition evaluates to true, the operator returns `expression_if_true`; otherwise, it returns `expression_if_false`. This is useful for simple conditional assignments without writing a full if-else statement.

Example:

```
int a = 10, b = 20;
int max = (a > b) ? a : b; // max = 20
```

2. Bitwise Operators

Bitwise operators operate directly on the individual bits of integer or character types. They are useful for tasks such as masking, toggling bits, or combining flags.

- `&` *(bitwise AND)*: sets each bit in the result to 1 only if both corresponding bits in the operands are 1.
- `|` *(bitwise OR)*: sets each bit in the result to 1 if at least one corresponding bit in the operands is 1.
- `^` *(bitwise XOR)*: sets each bit in the result to 1 only if the corresponding bits in the operands differ.
- `~` *(bitwise NOT)*: inverts all bits of a single operand.

Example:

```
int a = 5;  // 0101 in binary
int b = 3;  // 0011 in binary

int c = a & b; // 0001 → 1
int d = a | b; // 0111 → 7
int e = a ^ b; // 0110 → 6
int f = ~a;    // 1010... (inverted bits)
```

3. Increment & Decrement Operators

These operators increase or decrease a variable by one and come in two forms:

- Pre-increment/decrement: `(++a, --a)` – the variable is updated first, then the expression evaluates to the new value.
- Post-increment/decrement: `(a++, a--)` – the expression evaluates to the current value first, then the variable is updated.

Example:

```
int x = 5;
int y = ++x; // x = 6, y = 6
int z = x--; // x = 5, z = 6
```

The distinction between pre- and post-forms is important when using them in expressions, function arguments, or loops.

4. Modulus Operator

Apart from the basic arithmetic operations (`+, -, *, /`), C++ provides the modulus operator `%`, which returns the remainder after dividing one integer by another. It is useful for checking divisibility, cycling through ranges, or wrap-around calculations.

Example:

```
int a = 17;
int b = 5;
int remainder = a % b; // remainder = 2
```

5. Accumulative Operators

Compound assignment operators combine arithmetic operations with assignment. They make code shorter and easier to read

Operator	Description	Equivalent Expression
`+=`	Adds and assigns	`a += b → a = a + b`
`-=`	Subtracts and assigns	`a -= b → a = a - b`
`*=`	Multiplies and assigns	`a *= b → a = a * b`
`/=`	Divides and assigns	`a /= b → a = a / b`
`%=`	Modulus and assigns	`a %= b → a = a % b`

Example:

```
int a = 10;
a += 5;  // a = 15
a *= 2;  // a = 30
```

Lab Tasks

(Please Read instructions written on Page 1 before starting)

Task 1: Write a program that takes two integer inputs from the user and swaps their values without using a third variable. Display the swapped values.

Task 2: Write a program that requests the user's fullname as input and then asks for their age. Display a message that includes their name and age.

```
fall25/Lab_Solutions/lab02 on  main
❯ g++ task2.cpp && ./a.out
Enter your full name: Syed Taha
Enter your age: 100

Hello Syed Taha, you are 100 years old.
```

Task 3: When monitoring environmental conditions, it's often important to classify temperature readings into simple categories for quick assessment. For instance, a sensor might report a temperature in degrees Celsius, and we need to determine whether the environment is considered **Cold**, **Moderate**, or **Hot** based on predefined thresholds. This helps in systems like automated climate control, weather monitoring stations, or smart home devices.

Write a C++ program that:

- Prompts the user to enter the current temperature in Celsius.
- Checks the value and prints one of the following messages:
  - **Cold** if the temperature is below 15°C.
  - **Moderate** if the temperature is between 15°C and 25°C (inclusive).
  - **Hot** if the temperature is above 25°C.

Make sure your program reads the input only once and produces the correct output for any valid numeric temperature.

```
fall25/Lab_Solutions/lab02 on  main took 22s
❯ g++ task3.cpp && ./a.out
Enter the current temperature in Celsius: 37.2
Hot
```

Task 4: In many software systems, users have different access rights, such as the ability to **read**, **write**, or **execute** files. Each permission can either be **enabled** or **disabled**. Efficiently representing and checking these permissions is important in operating systems, file managers, and security applications.

Write a C++ program that:

- Prompts the user to enter a number between **0** and **7** representing the current permissions. The number is interpreted in binary as follows:

Number	Binary	Permission
0	000	No permissions
1	001	Execute
2	010	Write
3	011	Execute + Write
4	100	Read
5	101	Read + Execute
6	110	Read + Write
7	111	Read + Write + Execute

- Prints the status of each permission (**True** for enabled, **False** for disabled) based on the entered number.

- Prompts the user to enter a new **toggle mask** (a number between 0 and 7). Each bit in this number indicates which permissions to toggle: if a bit is set, the corresponding permission flips; otherwise it remains unchanged.
- Calculates the new permission states and prints them.

Hint: *Think in terms of binary operations!*

```
fall25/Lab_Solutions/lab02 on  main
❯ g++ task4.cpp && ./a.out
Enter current permissions (0-7): 5

Current Permissions:
Read  : True
Write : False
Exec  : True

Enter toggle mask (0-7): 7

New Permissions after toggling:
Read  : False
Write : True
Exec  : False
```

Task 5: In cryptography, one of the simplest ways to hide information is by combining it with a secret key using **binary operations**. Each character is stored as a number internally, so you can manipulate its bits to encode or decode it. A useful property of some operations is that **applying the same operation twice with the same key restores the original value**. This is the principle behind many simple encryption schemes.

Write a C++ program that:

- Prompts the user to enter a **single character** as the **secret**.
- Prompts the user to enter a **single character** as the **key**.
- Applies the key to the secret to produce an **encrypted character** and prints it.

Then, prompt the user to enter a **key** for decryption.

- If applying this key to the encrypted character reproduces the original secret, print the **original secret**.
- Otherwise, print **"Key error"**.

```
fall25/Lab_Solutions/lab02 on  main
❯ g++ task5.cpp && ./a.out
Secret: R
Key: L

Encrypted character: >

Enter key: L
Decrypted character: R
```

Think about what binary operation could allow you to encode and decode with the **same key**. There can be multiple ways to achieve this.